

The Science of a Transaction Representation Model, Layer by Layer

A faithful replication of Raman, Ganesh and Veloso, *Scalable Representation Learning for Multimodal Tabular Transactions* (arXiv:2410.07851, NeurIPS 2024), on synthetic ISO 20022 (pacs.008) payments.

Abstract. This paper documents the science of each layer of a faithful replication of the multimodal tabular-transaction method of arXiv:2410.07851. A power-law partitioned embedding encodes roughly 120,000 account identifiers in 5.8% of the parameters a dense table needs, with no loss of reconstruction accuracy (Layer 2, Sec. 3.1). A magnitude-adaptive, currency-conditioned quantizer turns amounts into tokens (3.3). An offline party encoder amortizes counterparty representation into a persistent store, so inference is a lookup rather than a forward pass (3.2). A bidirectional encoder is trained with a joint masked-reconstruction and batch-hard-triplet objective and then frozen (3.4). A frozen-encoder, frozen-LLM decoder learns four downstream tasks through about 7.64 million adapter parameters - 0.59% of a full fine-tune (Sec. 4). We state the two falsifiable claims the replication was built to test and report both, including the honest negative: on a synthetic, rule-derived risk label the adapter is extremely parameter-efficient but loses to a gradient-boosted tree, for reasons we trace to the data, not the architecture.

1. The problem, and the two planes

A payment is a wide, mixed-type record: high-cardinality identifiers (accounts, parents), a numeric amount spanning several orders of magnitude, categorical context, and counterparty attributes. Three properties make it hard to model. The identifiers have vocabularies in the hundreds of thousands. The interesting labels are rare - in our data the High-risk class is 1.8% of rows. And the data is regulated, so the representation has to be learned in a way that is auditable and cheap to serve.

The method answers this by splitting along a frozen-versus-trained boundary. An offline plane learns the expensive things once - the partitioned identifier space, the counterparty store, the encoder - and freezes them. An online plane scores a new payment with a store lookup and two frozen forward passes plus a small trained head. Everything below describes one stage of that pipeline, the science it rests on, and the design choices the paper left open. The deliverable of this replication is fidelity to the method, not improved metrics; where we depart, we say so.

2. Layer 1 - the input contract

Layer 1 is deterministic data engineering with no paper anchor: it parses each pacs.008 credit-transfer and projects it into a fixed-width row $x = (x_1, \dots, x_C)$ over C typed columns, then writes a bucket manifest (column_schema.json) that every later stage reads. Each column is typed into one of four buckets - high-cardinality categorical, numerical, core, and meta-party - and the bucket decides which Layer-2 encoder the column flows through. The contract matters more than it looks: pinning it to the schema, rather than to hard-coded column lists, is what lets a schema change reach every downstream module at once.

3. Layer 2 - field encoders

3.1 Partitioning embedder (paper Sec. 3.1)

A classical embedding stores one D -dimensional row per token, $E \in \mathbb{R}^{|V| \times D}$. For account identifiers with $|V|$ above 10^5 this table dominates the model's parameters, and most of it is barely trained - transaction identifiers follow a steep power law, so the long tail of rare accounts each appears a handful of times. Spending a full D -dimensional vector on a token seen three times is waste.

The fix is to size capacity to frequency. The vocabulary is split into B bins and the embedding dimension into B matching slices, each by a power law (Eq. 2 and its dimension analogue):

$$|V^b| = |V| \cdot b^{-a_v} / (\sum_{j=1..B} j^{-a_v}) \quad D^b = D \cdot b^{-a_d} / (\sum_{j=1..B} j^{-a_d})$$

With the paper's exponents $a_v = -3$ and $a_d = 2.25$, the negative a_v makes later bins hold far more tokens ($|V^1| \ll \dots \ll |V^B|$) while the positive a_d makes earlier bins far wider ($D^1 > \dots > D^B$). So the few frequent tokens land in a small, high-dimensional bin; the many rare tokens share a large, low-dimensional bin. The shared output space is a direct sum, $R^D = R^{D^1} (+) \dots (+) R^{D^B}$: a token assigned to bin b receives its D^b -dimensional row placed into bin b 's contiguous coordinate slice, and the other coordinates stay zero. There is no up-projection back to D - the partitioned subspaces are kept. The net parameter count is $\sum_b |V^b| \cdot D^b$, far below $|V| \cdot D$.

The paper does not state how tokens are assigned to bins; we assign by frequency rank, the only reading consistent with the power-law-on-frequency motivation, and flag the choice in the code. The claim under test (C1) is that this matches a dense table on reconstruction while using far fewer parameters. It holds: at a realized account vocabulary of 119,819, the partitioned tables use a 0.058 fraction of the dense parameter count and the masked-column reconstruction top-1 accuracy is 0.012 points *better* than the dense control, not worse.

3.2 Adaptive numerical quantization (Sec. 3.3)

Amounts are continuous and heavy-tailed. A uniform grid wastes most of its levels on the sparse high end and starves the dense low end of resolution. The method maps a numeric column to a custom vocabulary $Q = \{Q_1, \dots, Q_m\}$ that adapts to magnitude - finer spacing for small numbers, coarser for large - and assigns a value to its nearest level, $\text{argmin}_i |x - Q_i|$, which is then embedded like any categorical token. The paper leaves the spacing law and the level count open. We use geometric (log-spaced) levels, which give absolute gaps that grow with magnitude - exactly the stated behaviour, keyed to magnitude rather than to data density as a quantile grid would be - and $m = 128$ levels.

One forced departure lives here. Settlement amounts arrive in many currencies on wildly different scales (JPY versus USD), so a single global grid mis-quantizes multi-currency data. We build one grid per currency. The level-index vocabulary stays shared at size m , so level i means the i -th magnitude band *for its own currency*, and two currencies' level i share an embedding row; currency identity rides separately as a core column rather than being folded into the numeric token. This is a correctness fix, not an enhancement.

3.3 Offline party encoder and the persistent store (Sec. 3.2)

Counterparty attributes are meta-columns: contextual, slowly changing, and shared across many transactions. The method encodes them offline with a separate function $Xi_g : x_g \rightarrow R^{C_g \times D}$ whose output occupies a single element of the column sequence, with $C_g < |x_g|$ so the result is a compact pooled summary rather than a flattened block. The paper deliberately leaves the encoder, the pooling, and the training objective unspecified.

We encode the structured trio {Ctry, Industry, SubIndustry} with a small transformer over per-attribute embeddings plus a learned summary (CLS-style) token, and train it by masked-attribute reconstruction - mask one attribute, reconstruct it from the rest - which keeps the objective methodologically consistent with the Layer-3 loss. Party names are treated as identity, not as learnable attributes, because free-text names are high-cardinality identifiers already served by the Sec. 3.1 embedder through the account keys. The summaries are written once into a persistent store keyed by account id. At inference this store is a lookup, not a forward pass - the amortization that makes the online plane cheap, and the one place a careful reviewer will push, which is why the objective choice is documented rather than hidden.

3.4 Column assembler (Eq. 4)

The assembler routes each column to the encoder its bucket names and concatenates the results into the encoder's input sequence:

$$\text{embedding}(x) = (Xi(x_1), \dots, Xi_g(x_g), \dots, Xi(x_C))$$

High-cardinality columns get one partitioning embedder each - debtor and creditor accounts are kept separate, not merged into a shared vocabulary, which is the literal reading of Sec. 3.1. The party block is replaced by its pooled summary token, a frozen store lookup. Routing is driven entirely by the schema.

4. Layer 3 - the tabular encoder (Sec. 3.4)

A bidirectional transformer in the standard BERT configuration runs over the assembled column sequence; the output at a prepended [CLS] position is the row embedding $f(x)$. Training uses a composite objective that adds, to masked-column reconstruction, a batch-hard triplet loss:

$$L = L_{recon} + \lambda \cdot L_{triplet} \quad (\lambda = 1)$$

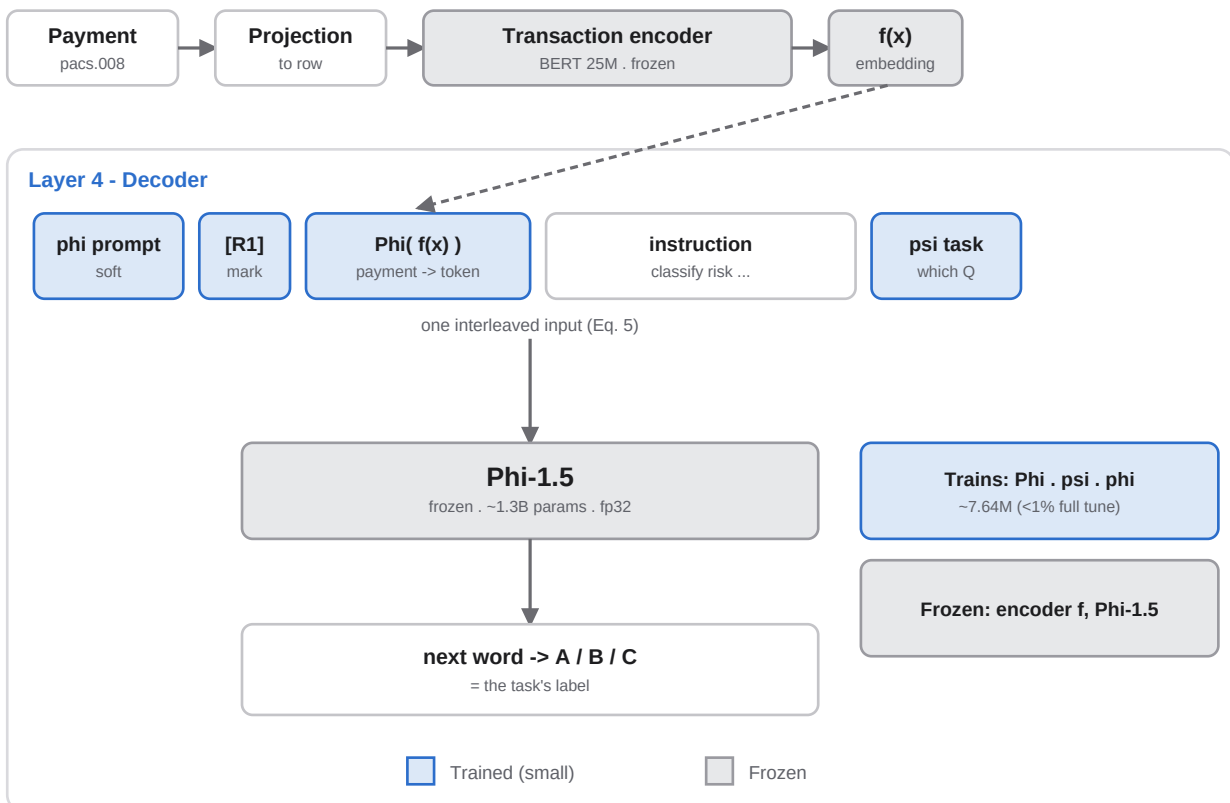
Reconstruction masks a subset of the reconstructable columns and predicts them with per-column heads (the frozen party-summary tokens serve as context and are not reconstructed). The triplet term is the part the paper defends in its ablation, and it is the claim under test, so it must not be swapped for a generic contrastive loss. We form two independently masked views of each row as a positive pair, and apply the batch-hard rule of Hermans et al. (2017): for every anchor, take the hardest positive (the farthest view of the same row) and the hardest negative (the nearest other row), and push them apart by a margin:

$$L_{triplet} = \text{mean over anchors of } \text{relu}(\text{margin} + d(\text{anchor}, \text{hardest_pos}) - d(\text{anchor}, \text{hardest_neg}))$$

The reconstruction loss teaches the encoder what each column says; the triplet loss shapes the geometry, pulling two corrupted views of one transaction together while pushing different transactions apart, so that distance in embedding space means something the per-column heads alone never enforce. The backbone is sized to the paper's pinned 25M parameters and trained for 3 epochs, the ablation sweet spot, then frozen. From here on f is a fixed feature extractor.

5. Layer 4 - the multimodal decoder (Sec. 4 / 4.1)

Layer 4 reads the frozen embedding with a frozen language model and learns the downstream tasks through three small trainable modules. The figure shows the runtime data flow.



The frozen tabular encoder f produces one embedding per payment. The frozen LLM is a Phi-1.5-class model. Only three modules train. The adapter Phi is a small transformer that projects $f(x)$ into the LLM's token-embedding space, emitting one soft token per record - this is what lets a frozen language model read a non-text object. The task embedding psi is a map $X_{i_task} : 1..K \rightarrow R^D$ built as $\text{concat}(\text{unique}[k], \text{shared})$: a subspace unique to each task and a subspace shared across all tasks. The prompt parameters phi augment the

model the way prompt-tuning does - either a per-layer learnable prefix (Li and Liang, 2021) or an input-level soft prompt, the robust default on a real HF model. The inputs to the LLM interleave a row sentinel, the payment token, the instruction text, and the task signal (Eq. 5):

$$z_i = X_{i_LLM}(s(1)) (+) \text{Phi}(f(x_{i,t})) (+) \dots (+) X_{i_LLM}(t_i) (+) X_{i_task}(k_i)$$

The row sentinels $s(.) = [R1], [R2], \dots$ uniquely identify records. Training is a single cross-entropy objective over the answer, supervised only at the response position (Eq. 6):

$$L = - \sum_i \log P(y_i | z_i; \text{Phi}, \text{psi}, \text{phi})$$

Single-record tasks pass one payment; the recurrence task passes several of an entity's payments together, repeating the sentinel-and-token block before the instruction, because recurrence is a pattern across records rather than a property of one. The whole trainable trio is about 7.64 million parameters against a 1.3-billion frozen LLM - 0.59%. Keeping f and the LLM frozen is both the deployability argument and the integrity of the experiment: unfreeze either and the headline numbers describe a different model.

6. Layer 5 - baselines and falsifiable evaluation

The baseline is CatBoost trained on the raw, flattened row - no representation learning, the floor the adapter must clear. The recurrence task is excluded from the CatBoost comparison, following the paper (Sec. 5.3): it passes multiple transactions and 'is not suitable for non-sequential classifiers'. Because the risk label is imbalanced, accuracy is misleading by construction, so the primary metrics are a forced departure: PR-AUC, recall at a fixed 1% false-positive rate, and F1 at the operating point; accuracy is reported alongside only for comparability to the paper's balanced-task tables.

The replication is organized around two falsifiable claims with thresholds set before the run. C1: the partitioning embedder matches dense reconstruction while using substantially fewer embedding parameters. C2: the frozen-encoder, frozen-LLM adapter beats CatBoost and rivals a full fine-tune at a fraction of the trainable parameters. A run passes only if the measured number clears its pre-registered threshold.

7. Results, stated honestly

Claim / metric	Measured	Threshold	Verdict
C1 embedding param ratio (partitioned / dense)	0.058	≤ 0.55	pass
C1 top-1 reconstruction gap (dense - partitioned)	-0.012 pp	≤ 1.0 pp	pass
C2 trainable param ratio (adapter / full tune)	0.0059 (7.64M / 1.3B)	≤ 0.10	pass
C2 risk PR-AUC, adapter vs CatBoost	0.21 vs 0.66	gain $\geq +10$ pp	fail (-44.5 pp)

C1 is confirmed at near-paper scale and is the headline replication: the partitioned embedder reaches the same reconstruction quality as a dense table at 5.8% of the embedding parameters. C2 is half-confirmed. The adapter is extremely parameter-efficient - it trains under 1% of a full fine-tune - but on the risk task it loses to CatBoost, and not narrowly.

The loss is a property of the data, not the architecture, and the distinction is the point. Our synthetic risk label is a transparent rule over cross-border status, currency, region, industry, and amount - the exact fields CatBoost reads off the row - so the tree recovers the rule. The adapter has to carry the same signal through a frozen encoder compressed to a single token and a head trained for one epoch. There is a general argument underneath this: for a single-record label that is a function of visible fields, CatBoost has strictly more direct access to those fields than a frozen, compressed embedding does, so the adapter can at best tie. A representation earns its advantage only where the flattened tabular view loses information - across records, and over time.

8. Limitations and where the science points

The result is honest about its own ceiling. On single-record, full-information tasks a gradient-boosted tree is near-optimal, so a representation model cannot win there. Its advantage lives in structure the tree cannot see: the timing between an entity's payments, a dormant account reactivating, a pattern that only reads as fraud across a history. The repository already carries the multi-record machinery for that case (recurrence, Eq. 5); what it does not carry is real transactions over time. That is the same direction the production transaction foundation models have taken - Visa's TransactionGPT (arXiv:2511.08939) and Revolut's PRAGMA (arXiv:2604.08649) both model behaviour as a sequence over time. The concrete next steps are temporal-sequence modelling over entity histories (TabBERT, FATA-Trans, CoLES) and a held-out-entity split that tests generalization rather than memorization.

9. Deviations ledger

Three departures from the paper are sanctioned and visible throughout: the pacs.008 column schema in place of the paper's generic synthetic columns; currency-conditioned quantization, a correctness fix for multi-currency amounts; and imbalance-aware metrics in place of accuracy, forced by the rare positive class. Three extensions are deliberately walked back and excluded: a data-completeness feature vector, a structuring/layering chain task, and a held-out-typology split. The recurrence task is a paper task and is in scope - it should not be confused with the walked-back chain task.

References

- Raman, B., Ganesh, S., Veloso, M. (2024). Scalable Representation Learning for Multimodal Tabular Transactions. arXiv:2410.07851. NeurIPS 2024 Table Representation Learning workshop.
- Hermans, A., Beyer, L., Leibe, B. (2017). In Defense of the Triplet Loss for Person Re-Identification. arXiv:1703.07737.
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT.
- Li, X. L., Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL. arXiv:2101.00190.
- Li, Y., et al. (2023). Textbooks Are All You Need II: phi-1.5 Technical Report. arXiv:2309.05463.
- Prokhorenkova, L., et al. (2018). CatBoost: Unbiased Boosting with Categorical Features. NeurIPS.
- Padhi, I., et al. (2021). Tabular Transformers for Modeling Multivariate Time Series (TabBERT). ICASSP. arXiv:2011.01843.
- Babaev, D., et al. (2022). CoLES: Contrastive Learning for Event Sequences with Self-Supervision. SIGMOD. arXiv:2002.08232.
- Zhang, D., et al. (2023). FATA-Trans: Field And Time-Aware Transformer for Sequential Tabular Data. CIKM. arXiv:2310.13818.
- Visa Research (2025). TransactionGPT. arXiv:2511.08939.
- Revolut (2026). PRAGMA: Revolut Foundation Model. arXiv:2604.08649.